Java OOP Laboratory Exercise

Campus Café Ordering System

Name: Cuadra, Vincent Paul K.

Section: D

Date: November 10, 2026

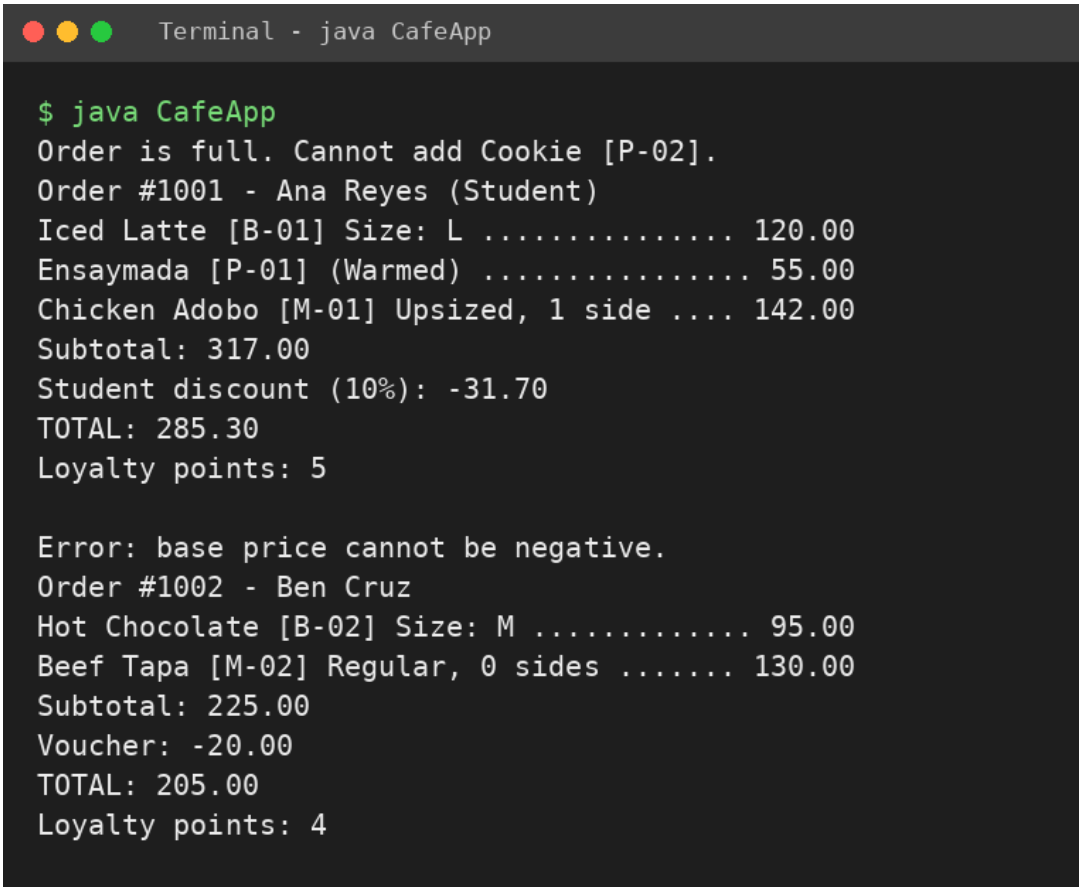
Exercise: Campus Café Ordering System

1. Concept Explanation

Concept	Where and why I used it
if Statement	Pastry.computePrice() adds the 10-peso warming fee and Meal.computePrice() adds the upsize fee, because each is a single yes/no decision.
if-else Statement	MenuItem.setBasePrice() rejects negative prices; Beverage.computePrice() uses an if-else-if chain for sizes; Order.addItem() checks the 3-item limit; Order.computeDiscount()/computeTotal() apply the student discount and keep the total at 0 or higher.
Classes and Objects	Seven classes (MenuItem, Beverage, Pastry, Meal, Customer, Order, CafeApp), each in its own file; CafeApp.main() creates the objects.
Attributes and Behaviors	Every class stores data in instance variables (e.g., size, warmed, loyaltyPoints) and acts on it with instance methods (e.g., computePrice(), addPoints()).
Object Interaction	Order stores a Customer and up to three MenuItem objects; printReceipt() calls each item's methods and then updates the Customer's loyalty points.
Naming Conventions	PascalCase for classes, camelCase for methods and variables, UPPER_SNAKE_CASE for constants such as MAX_ITEMS and STUDENT_DISCOUNT_RATE.
Default Constructor	MenuItem(), Beverage(), Pastry(), Meal(), and Customer() give safe starting values so objects can be built first and filled in later with setters.
Parameterized Constructor	All MenuItem subclasses, Customer, and Order accept values at creation so an object is complete right away; subclasses pass shared data up with super(...).
Constructor Overloading	Customer has three constructors (none, name, name + student) that chain to one another with this(...).
public / private / protected	Fields are private, except name and basePrice in MenuItem, which are protected so subclasses can use them in pricing; methods are public.
Getters, Setters, Data Hiding	No public fields. Setters validate input (no negative price, size only S/M/L, no negative sides or loyalty points, no blank names).
Method Overloading	Customer.addPoints(int) / addPoints(double) and Order.computeTotal() / computeTotal(double); I also overloaded printReceipt() to show a voucher line.
Method Overriding	computePrice() and getDescription() are overridden with @Override in Beverage, Pastry, and Meal so each type prices and describes itself.
Abstraction	MenuItem is abstract with the abstract method computePrice(), so it cannot be instantiated and every subclass must supply its own pricing rule.

2. Program Output

Output of `java CafeApp`. Order #1001 matches the expected values (subtotal 317.00, discount -31.70, total 285.30, 5 points) and Order #1002 as well (subtotal 225.00, voucher -20.00, total 205.00, 4 points). The "Order is full" message and the negative-price error show the validation working.



3. Challenges Encountered and Solutions

- Storing three items without an ArrayList: I used three fields (item1, item2, item3) and an itemCount. addItem() fills the first empty slot with an if-else-if chain, and computeSubtotal() checks each slot for null before calling computePrice().
- Awarding points only once: printReceipt() could be called more than once and would give duplicate points, so I added a private pointsAwarded flag in Order.
- Showing a voucher on the receipt: the required printReceipt() takes no parameters, so I added an overloaded printReceipt(double voucher) and made the original call it with 0.
- Keeping validation consistent: constructors call the same validated setters (e.g., setBasePrice, setSize, setExtraSides), so invalid data is rejected whether an object is built with a constructor or with setters.
- Compiling without an IDE: I compiled and ran everything with plain javac and java CafeApp from the project folder to meet the submission rules.